



Trabajo Práctico Diseño: Money Laundering Analysis

Ingeniería en Informática — Año 2026



Integrantes

Nombre y Apellido	Legajo	Correo electrónico
Juan Martín de la Cruz	109588	jdelacruz@fi.uba.ar
Ian Klaus von der Heyde	107638	ivon@fi.uba.ar
Agustín Altamirano	110237	aaltamirano@fi.uba.ar

Índice

1. Introducción	3
2. Escenarios	3
3. Vista Lógica	4
3.1. Grafo Acíclico Dirigido (DAG)	4
4. Vista de Procesos	6
4.1. Diagrama de Secuencia	6
4.2. Diagramas de Actividad	7
5. Manejo de EOF	16
5.1. Topología de anillo	16
5.2. Garantía de entrega confiable	17
5.3. Trade-offs	17
6. Vista Física y de Despliegue	18
6.1. Diagrama de Robustez	18
6.2. Diagrama de Despliegue	21
7. Vista de Desarrollo	22
7.1. Diagrama de Paquetes	23
8. Planificación y Organización	24
8.1. Listado de Tareas a Ejecutar	24
8.2. División del Trabajo entre Integrantes	25

1. Introducción

El presente documento detalla la arquitectura y el diseño de un sistema distribuido cuyo propósito es analizar extractos de transacciones entre cuentas bancarias para detectar anomalías. Este análisis se enmarca en la detección del lavado de activos, un delito que consiste en cambiar bienes o dinero obtenidos mediante actos ilícitos por dinero legítimo. Para lograr esto, los infractores utilizan la red digital de pagos aplicando diversas estrategias de ofuscación, tales como transacciones Fan-out, Fan-in, patrones Scatter-Gather, grafos Bipartitos y Stack.

Para hacer frente a esta problemática, se diseñó una solución optimizada para entornos multi-computadoras. La arquitectura propuesta permite escalar los elementos de cómputo horizontalmente para soportar el incremento en los volúmenes de información y clientes simultáneos a procesar. A nivel de infraestructura, el sistema se apoya en un Middleware que abstrae la comunicación basada en grupos. Asimismo, el flujo de procesamiento soporta una única ejecución continua y provee mecanismos de cierre ordenado.

Los datos utilizados para el procesamiento provienen del dataset de simulaciones bancarias de IBM (disponible en Kaggle.), integrándose además con la API Frankfurter para obtener las cotizaciones diarias necesarias para las conversiones de divisas a USD.

2. Escenarios

Query 1 — Micropagos (Fan-out / Fan-in): Se buscan todas las transacciones en dólares estadounidenses (USD) cuyo monto sea estrictamente menor a \$50. Para cada una se reportan la cuenta de origen, la cuenta de destino y el monto transferido. Montos pequeños y frecuentes son un indicador típico de los patrones *fan-out* y *fan-in*, donde una cuenta dispersa o concentra fondos en pequeñas fracciones para evitar los umbrales de reporte automático de las entidades bancarias.

Query 2 — Máximos por Entidad Bancaria: Para cada banco de origen presente en el conjunto de transacciones en USD, se identifica la transferencia de mayor monto. El resultado incluye el nombre del banco, la cuenta emisora y el monto correspondiente. Esta consulta permite detectar cuentas que concentran movimientos de alto valor dentro de una misma entidad, lo cual puede ser indicativo de operaciones atípicas o de una cuenta central en un esquema de lavado.

Query 3 — Anomalías Históricas (Stack): Se analiza el período [2022-09-06, 2022-09-15] comparándolo con el período histórico de referencia [2022-09-01, 2022-09-05]. Para cada formato de pago, se calcula el promedio de montos en USD del período de referencia. Luego se reportan las transacciones del período analizado cuyo monto sea inferior a un centésimo (1 %) de dicho promedio. Este tipo de anomalía corresponde al patrón *stack*: transferencias acumuladas de montos ínfimos a lo largo del tiempo, diseñadas para pasar desapercibidas.

Query 4 — Patrón Scatter-Gather: Se identifican las cuentas que participan en un esquema de *scatter-gather* con exactamente una cuenta intermediaria. La condición es que la cuenta de origen haya realizado transferencias en USD hacia al menos 5 cuentas intermediarias distintas durante el período [2022-09-01, 2022-09-05], y que cada una de esas cuentas intermediarias haya transferido fondos hacia una misma cuenta de destino final. Este patrón describe el flujo *fan-out* desde un origen, seguido de un *fan-in* hacia un destino, utilizando cuentas intermediarias.

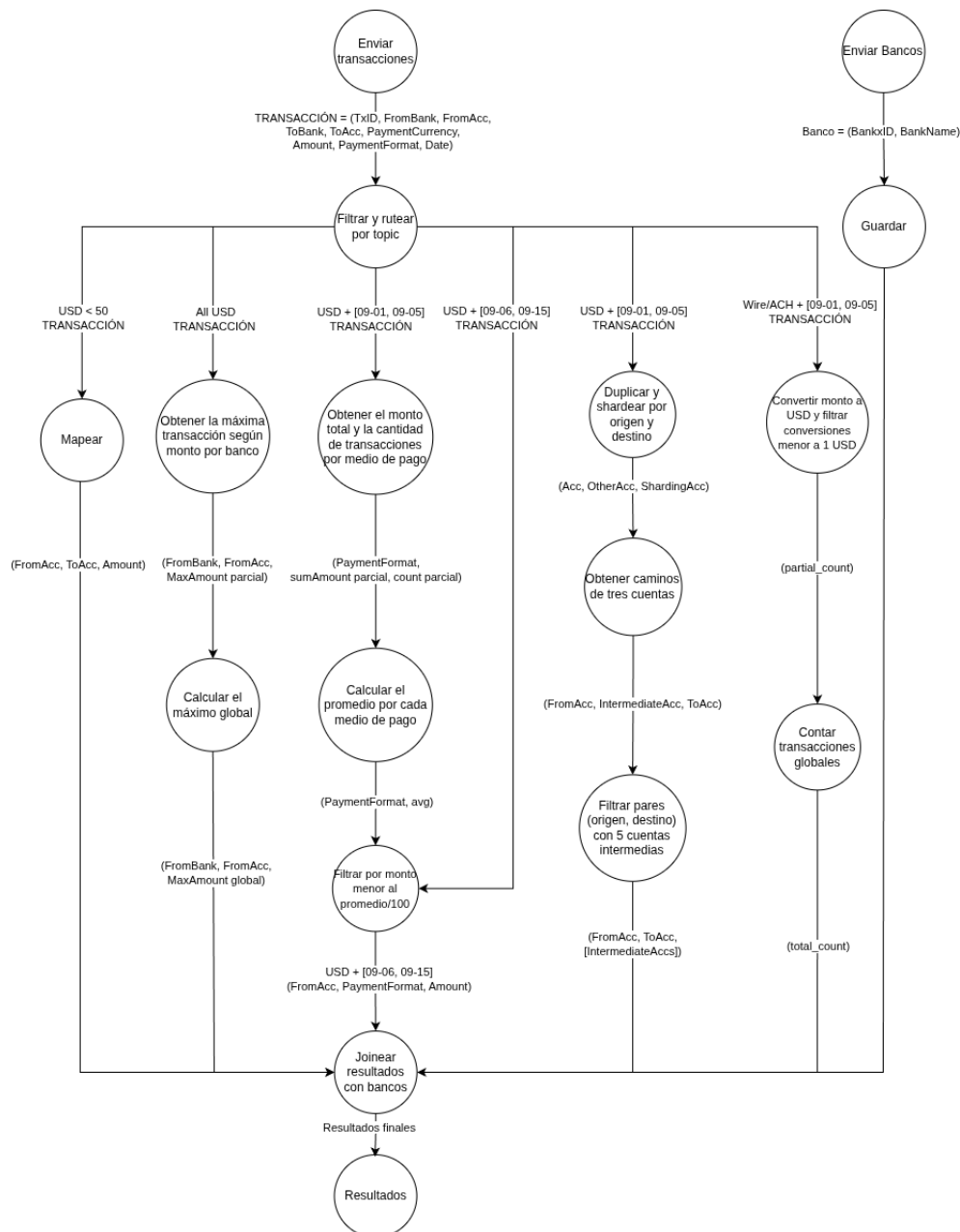
Query 5 — Micropagos Internacionales (Wire / ACH): Se contabilizan las transacciones del período [2022-09-01, 2022-09-05] realizadas mediante los formatos de pago *Wire* o *ACH* cuyo monto, una vez convertido a USD utilizando el tipo de cambio diario provisto por la API de Frankfurter, sea estrictamente menor a \$1. La conversión dinámica permite detectar micropagos internacionales que, expresados en moneda local, pueden no parecer sospechosos pero que en USD revelan un volumen anómalo de operaciones de bajísimo valor.

3. Vista Lógica

En esta sección presentamos la vista lógica del sistema, cuyo objetivo es describir el flujo de datos desde los archivos fuente enviados por el Cliente (**Trans.csv**, **accounts.csv**) hasta el sumidero final (los resultados consolidados que se devuelven al mismo cliente). El énfasis está puesto en las transformaciones que sufre cada dato, sin todavía comprometernos con cómo se distribuyen físicamente esas transformaciones entre nodos.

3.1. Grafo Acíclico Dirigido (DAG)

A continuación presentamos el DAG que representa el flujo lógico de los datos del sistema. Cada nodo del grafo es una transformación lógica y cada arista lleva la tupla concreta de datos que se transmite entre etapas a través de un *exchange* o *queue* de RabbitMQ. La distinción entre los distintos tipos de nodos es importante para anticipar los requisitos de escalabilidad y coordinación de cada etapa.



A grandes rasgos, el ciclo de vida de un dato consiste en la fuente **Enviar transacciones** produce una tupla por cada transacción del archivo de entrada; un **router** central que inspecciona la tupla y la distribuye, según su contenido, a una o más ramas Q1–Q5. Cada rama aplica las transformaciones específicas necesarias para resolver su consulta y emite resultados parciales hacia el *Final Joiner*. A su vez, la fuente de datos de Bancos produce una tupla con el identificador único y nombre del banco, que se transmite al *Final Joiner*. Una vez que ambos flujos de datos llegan ahí, se consolidan los resultados finales.

Tipos de nodos lógicos

Para comprender la topología del grafo, resulta indispensable categorizar los nodos según su manejo de estado. Por ello decidimos agruparlos y asignarles un color a cada uno:



- **Map / Filter:** Operan de forma puramente funcional sobre cada transacción individual. No requieren memoria de eventos pasados. Emiten los resultados de cada dato inmediatamente. Ejemplos: Dispatcher (Q1), Period Filter (Q3), Sharder (Q4) y Converter (Q5).
- **Reducer:** Mantienen un estado local mientras procesan un flujo continuo de datos de un topic general. Ejemplos: Máximo local (Q2), Suma local (Q3) y 3-Path Finder (Q4).
- **Global Aggregator:** Consumen resultados parciales previamente enrutados (shardeados) por una clave de negocio. Al tener garantizada la exclusividad de los datos de su partición, no requieren coordinación en anillo con sus pares; simplemente totalizan y emiten el resultado final al recibir las señales de cierre de las etapas previas. Ejemplos: Max global (Q2), Promedio (Q3), Conteo de Intermediarios (Q4) y agrupación (Q5)

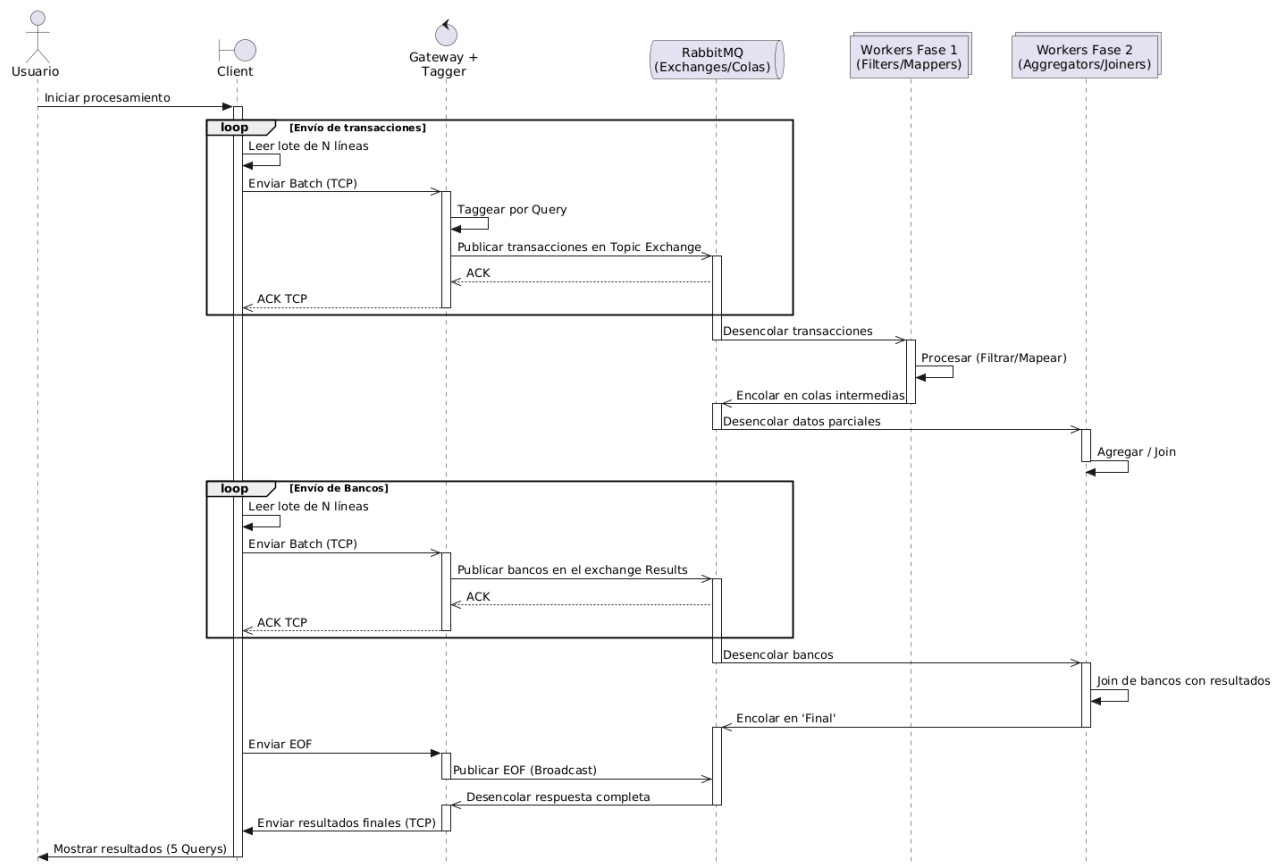
Es importante aclarar que los nodos de entrada de cada consulta (sean Map / Filter o Reducers) son los encargados de la coordinación del EOF mediante la topología de anillo (descrito en la sección *Manejo de EOF*) y de aplicar la clave de sharding correspondiente para derivar los datos a las etapas posteriores.

4. Vista de Procesos

En la presente sección describimos la dinámica del sistema, es decir, cómo interactúan los distintos componentes a lo largo de la ejecución y cómo se coordinan para producir los resultados de cada consulta. Presentamos en primer lugar un diagrama de secuencia que ilustra las interacciones entre las entidades principales, y a continuación los diagramas de actividad que detallan el comportamiento interno de cada componente.

4.1. Diagrama de Secuencia

El diagrama que sigue resume el flujo completo de una sesión de cliente: el establecimiento de la conexión TCP, el envío de *batches* de transacciones y bancos, la propagación de éstas hacia las distintas *Topic Queues*, el procesamiento por las consultas Q1–Q5, la convergencia en el *Final Joiner* y la entrega final de resultados al cliente.

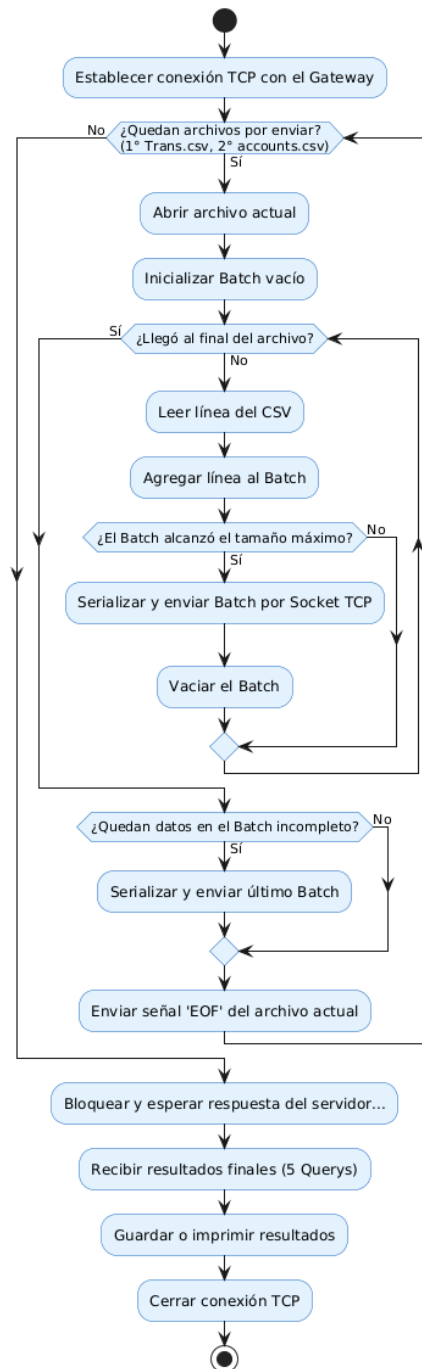


Resulta importante destacar que la comunicación entre cliente y *gateway* es síncrona y orientada a la conexión (TCP/IP), mientras que toda la comunicación interna entre el *gateway* y los *workers* es asíncrona y mediada por RabbitMQ. Esta separación permite desacoplar la velocidad de ingreso de los datos del cliente de la velocidad de procesamiento del *backend*, y aislar al cliente de los detalles de coordinación interna del sistema.

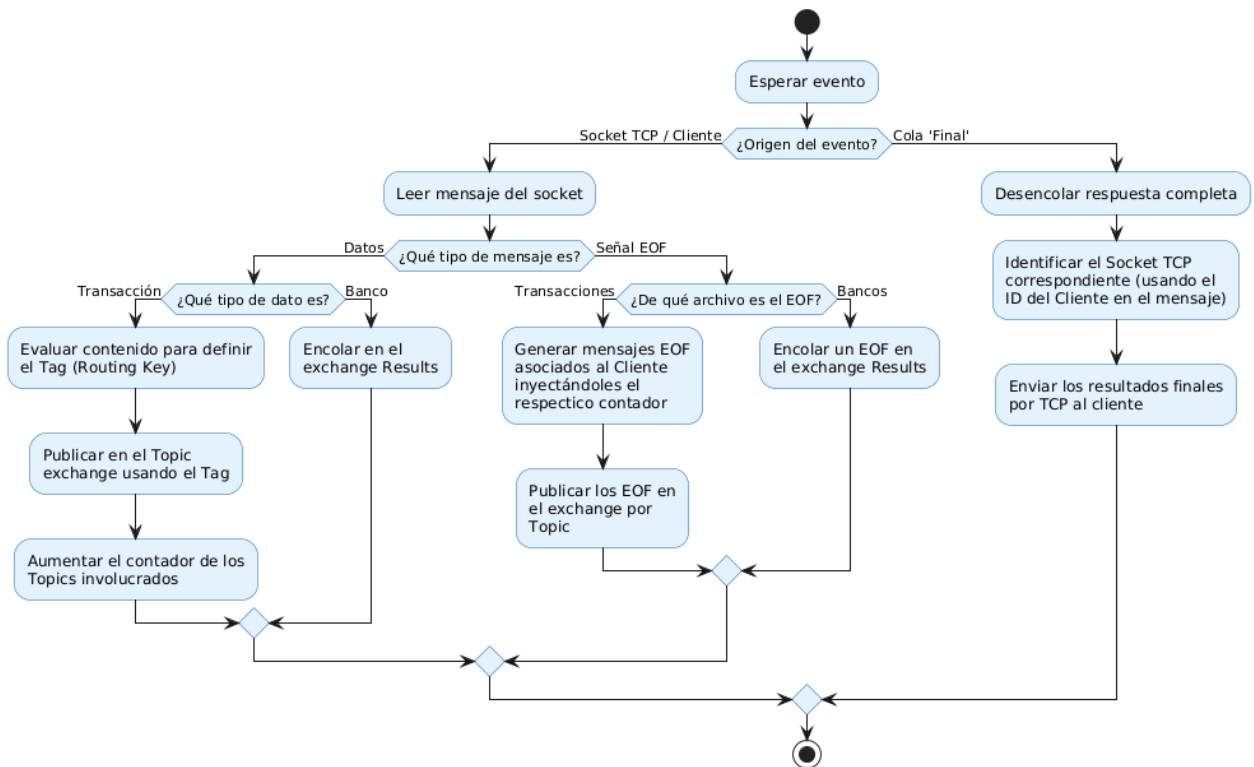
4.2. Diagramas de Actividad

A continuación se presentan los diagramas de actividad de los distintos componentes del sistema. Cada uno describe el procesamiento que un *worker* realiza por cada mensaje recibido, así como su comportamiento al detectar la señal de fin de procesamiento (EOF). El protocolo concreto mediante el cual se garantiza que el EOF refleja el cierre de los datos del cliente se describe en la sección *Manejo de EOF*; en lo que sigue asumimos que esa garantía existe.

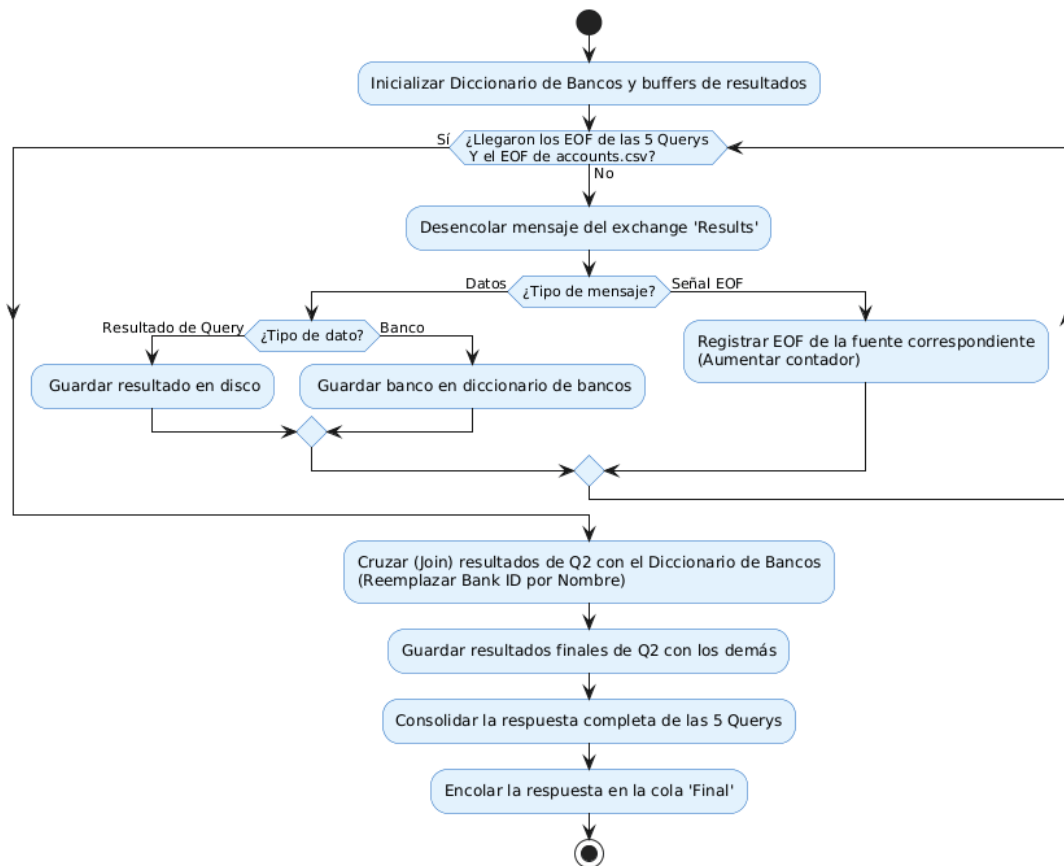
- **Cliente.** El cliente abre una conexión TCP con el *gateway*, lee el archivo `Trans.csv` línea por línea y arma *batches* de tamaño configurable que envía por el socket. Al alcanzar el final del archivo serializa el último *batch* y emite la señal EOF sobre el mismo socket. Una vez hecho esto, sigue exactamente los mismos pasos para el archivo `accounts.csv`. A continuación se bloquea a la espera de los resultados, los recibe en un único bloque consolidado y cierra la conexión.



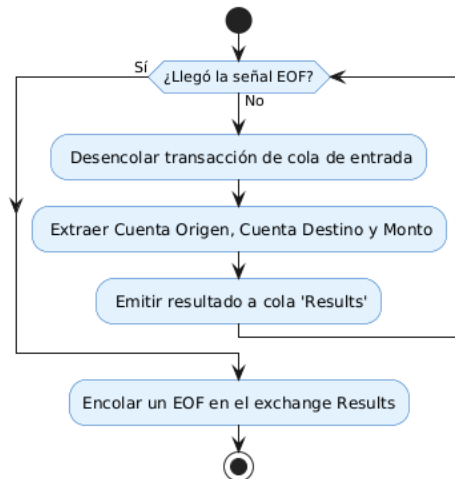
- **Gateway + Tagger.** El *gateway* actúa como un selector de eventos: cada iteración lee del socket TCP del cliente o de la cola *Final* de RabbitMQ. Si el evento proviene del cliente y se trata de una transacción, evalúa el contenido de la misma y le asigna un *routing key* acorde a las consultas a las que aplica, publicándola en el *Topic Exchange*. Si proviene del cliente y es un banco, lo encola directamente en el exchange Results. Adicionalmente lleva un contador de filas por *topic*, de modo que al recibir el EOF de transacciones del cliente pueda generar los mensajes de EOF hacia cada cola con el conteo total de filas que ese *topic* debió procesar (esta cifra es la que utiliza el protocolo de anillo). Si el evento proviene de la cola *Final*, identifica el cliente destinatario por el *ClientID* contenido en el mensaje y envía los resultados por el socket correspondiente.



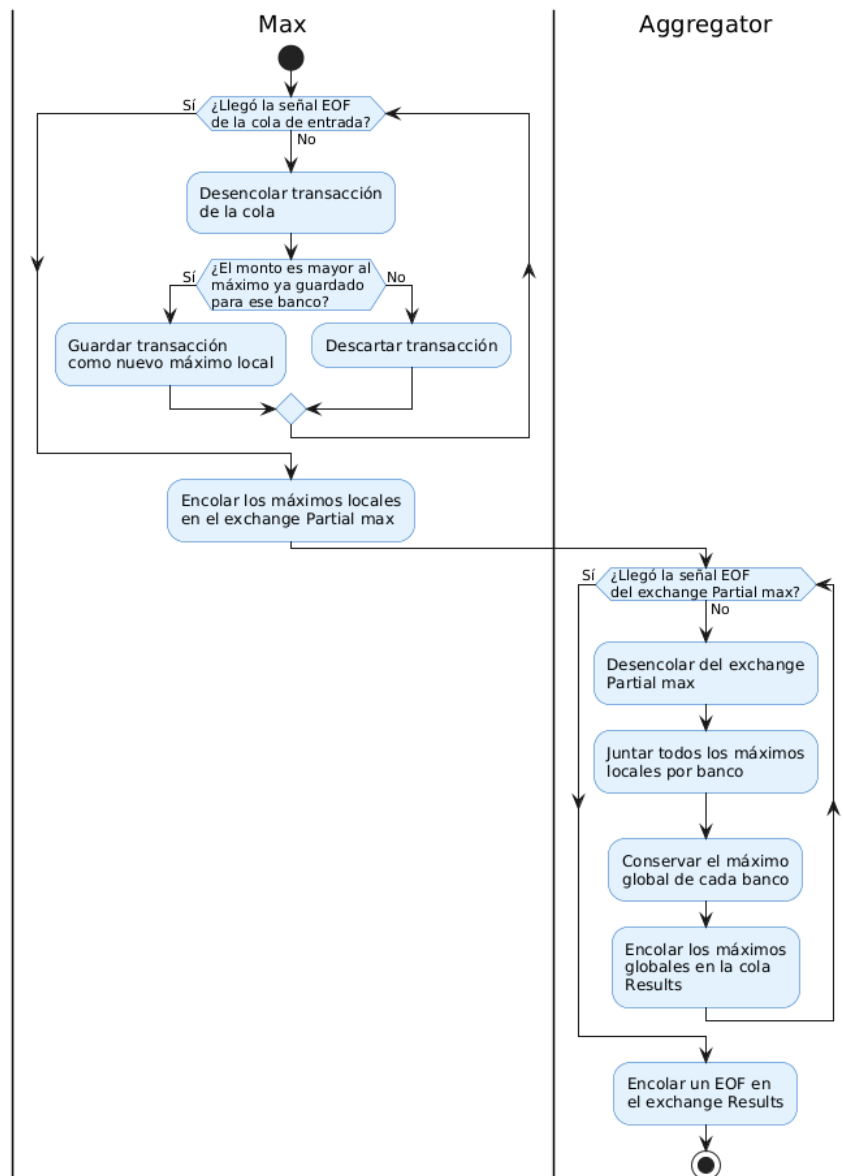
- **Final Joiner.** Consume del *exchange Results*, donde convergen los resultados de las cinco consultas y de los bancos. Para cada cliente mantiene un estado parcial al que va incorporando las respuestas de cada *topic*, y a su vez mantiene en memoria el identificador y nombre de cada banco. Cuando ha recibido los EOFs de las cinco consultas (uno por cada rama Q1–Q5) y el EOF de bancos, considera al cliente listo y emite la respuesta consolidada hacia la cola **Final**, desde donde el *gateway* la recogerá para enviarla al cliente.



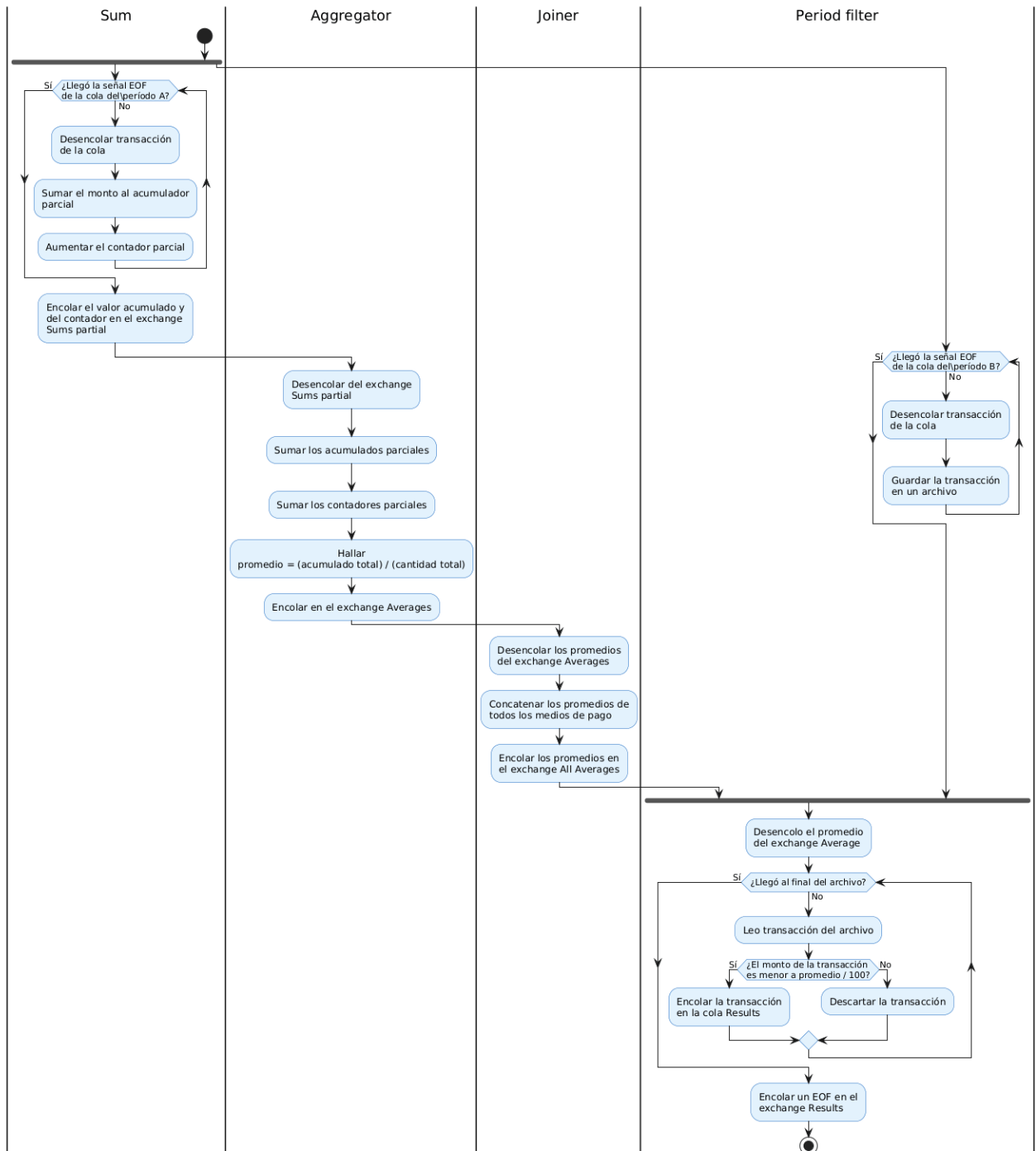
- **Query 1 (Micropagos).** Es la consulta más simple. El único *worker* de la rama opera de manera *online*: por cada transacción que recibe extrae los datos (**FromAcc**, **ToAcc**, **Amount**) y la encola directamente en **Results**. Al recibir el EOF lo propaga a **Results** y termina su iteración para ese cliente.



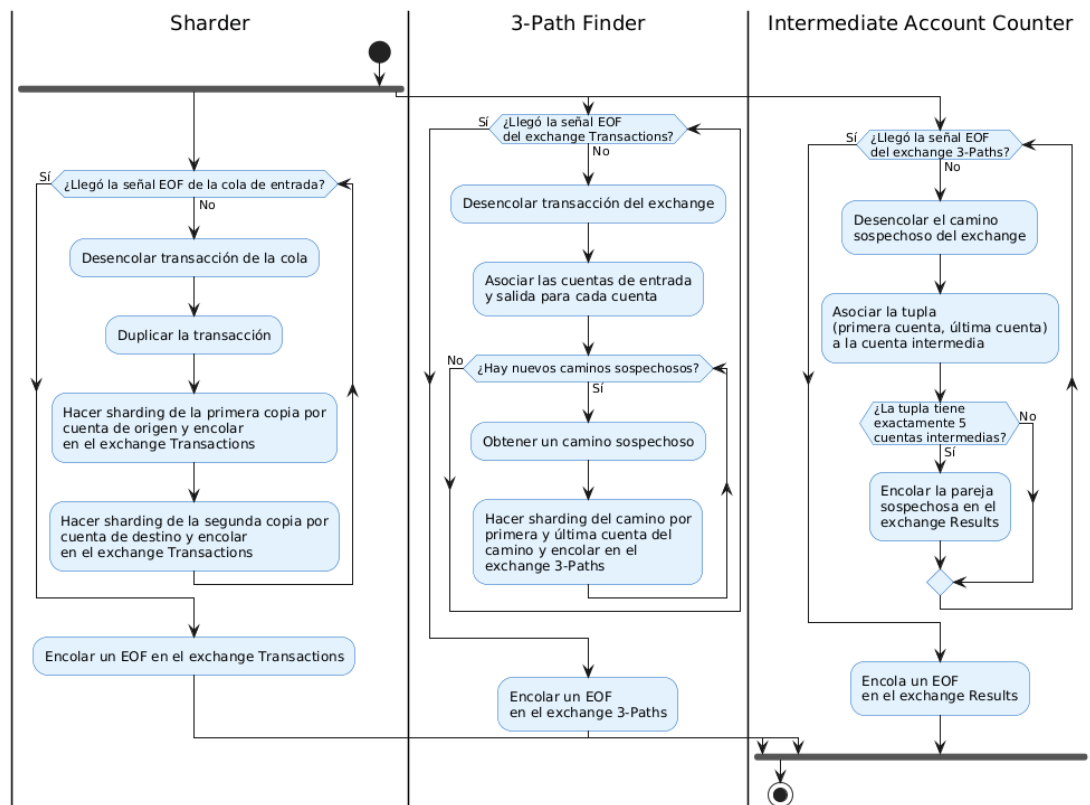
- **Query 2 (Máximos por Entidad Bancaria).** Se compone de dos etapas. La primera, *Max*, mantiene un máximo local por banco mientras consume transacciones del *Topic Queue* correspondiente; al recibir el EOF flushea su tabla parcial al *exchange Partial max*. La segunda, *Aggregator*, consume los máximos parciales y conserva el máximo global por banco, emitiendo el resultado a **Results** junto con su EOF.



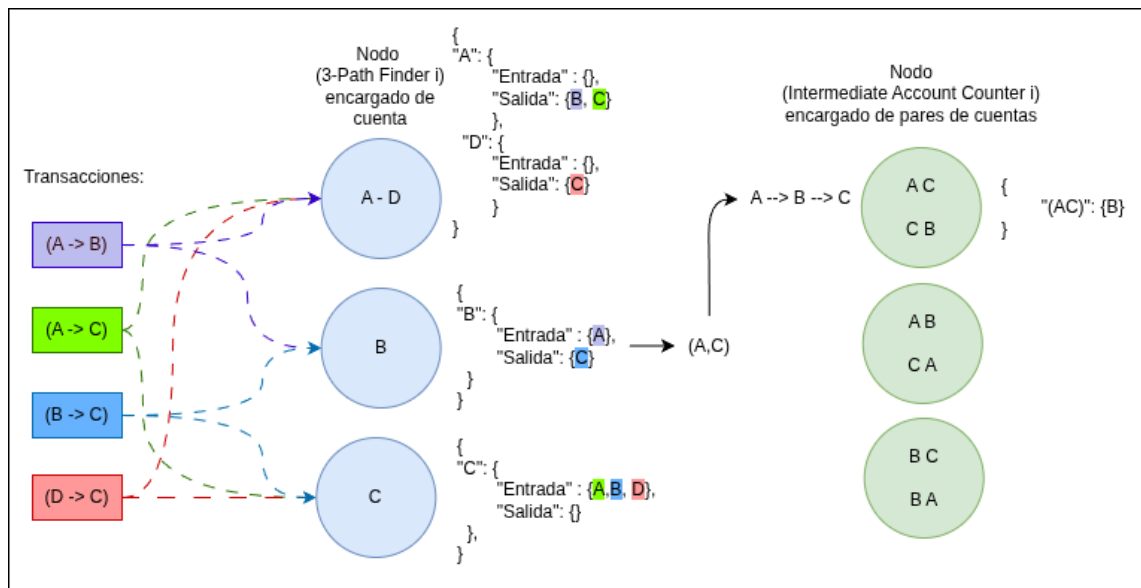
- Query 3 (Anomalías Históricas).** Esta consulta tiene dos sub-ramas concurrentes. La sub-rama de *Sum* recibe transacciones del período de referencia y mantiene, por formato de pago, un acumulador y un contador parciales que al recibir EOF emite al *exchange Sums partial*. El *Aggregator* consume estos parciales, calcula el promedio por formato de pago y lo publica en el *exchange Averages*. El *Joiner* simplemente se encarga de concatenar todos los promedios de cada medio de pago calculados en un único mensaje y mandarlo al exchange *All averages*. En paralelo, el *Period filter* recibe transacciones del período analizado y las persiste a disco a la espera del promedio. Una vez disponible, recorre el archivo y emite a **Results** solamente las transacciones cuyo monto sea inferior a un centésimo del promedio del formato de pago correspondiente, obtenido del mensaje desencolado del exchange *All averages*.



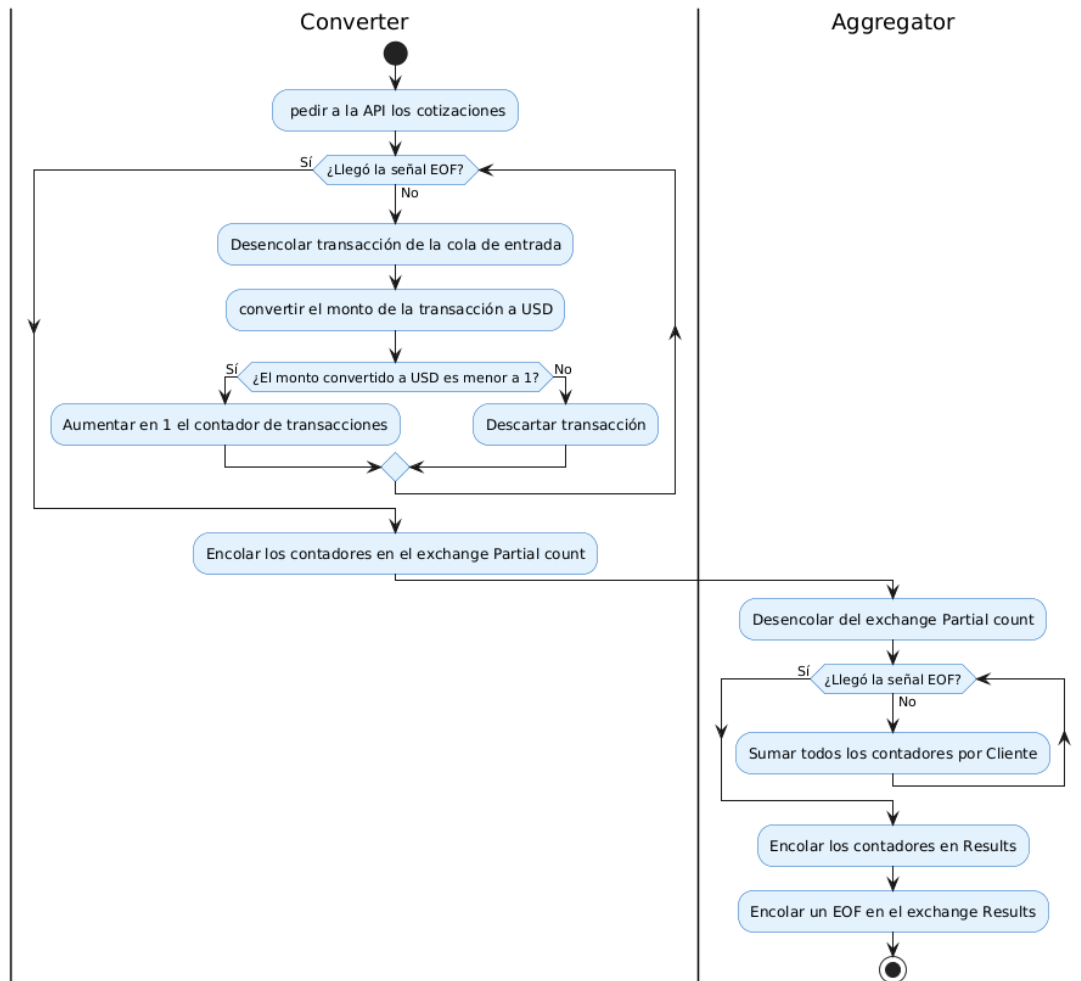
- **Query 4 (Scatter-Gather).** La rama está compuesta por tres etapas en *pipeline*. El *Sharder* recibe cada transacción y emite dos copias *shardeadas*: una por cuenta de origen y otra por cuenta de destino, ambas hacia el *exchange Transactions*. El *3-Path Finder* consume del *exchange Transactions* y, para cada cuenta intermedia, mantiene los conjuntos de cuentas que le envían fondos y a las que ella envía fondos; cada vez que descubre un nuevo camino sospechoso de la forma (origen, intermedia, destino), lo *shardea* por el par (origen, destino) y lo emite al *exchange 3-Paths*. Finalmente, el *Intermediate Account Counter* agrupa los caminos por par y emite a *Results* aquellos pares (origen, destino) que poseen al menos cinco cuentas intermedias distintas.



Para hacer tangible el algoritmo de Q4, el siguiente esquema muestra una ejecución de ejemplo. A la izquierda aparecen las transacciones de entrada, y en el centro la representación interna del *3-Path Finder* para cada cuenta intermedia: por cada cuenta se mantienen los conjuntos **Entrada** (cuentas que le envían fondos) y **Salida** (cuentas a las que envía fondos). Cada vez que la cuenta acumula un nuevo elemento en **Entrada** o en **Salida**, se emiten todos los nuevos caminos que se pueden armar con ese nuevo elemento. A la derecha se observa cómo el *Intermediate Account Counter* agrupa los caminos por el par (origen, destino) y mantiene el conjunto de cuentas intermediarias asociadas a cada par.



- **Query 5 (Micropagos Internacionales).** La rama está compuesta por dos etapas. La primera, *Converter*, obtiene al iniciar las cotizaciones diarias de la API Frankfurter, consume transacciones del *Topic Queue* correspondiente, convierte el monto a USD y, si resulta inferior a 1, incrementa un contador parcial. Al recibir EOF emite ese contador al *exchange Partial count*. La segunda etapa, *Aggregator*, suma los contadores parciales por cliente y emite el total a *Results*.



5. Manejo de EOF

Antes de avanzar con la vista física, resulta necesario describir el mecanismo mediante el cual el sistema garantiza el cierre ordenado del procesamiento de cada cliente. Dado que cualquier consulta involucra grupos de *workers* idénticos consumiendo en paralelo de una misma cola, la simple llegada de un mensaje de EOF a uno de ellos no alcanza para asegurar que todos los datos del cliente hayan sido efectivamente procesados: aún pueden existir mensajes en tránsito, en *buffers* internos o en proceso de ser *ackeados*. Adoptar una política optimista en este punto comprometería la corrección de los resultados, lo cual es inadmisibles para una aplicación cuyo dominio es la detección de patrones en transacciones financieras.

5.1. Topología de anillo

Para resolver este problema se adoptó una topología de anillo entre los *workers* de cada grupo. Cuando mencionamos que un conjunto de N *workers* idénticos utiliza topología de anillo para el procesamiento de la señal EOF, cada *worker* i tiene una cola privada **worker_ring_i** de la que consume y publica a la cola **worker_ring_{-(i+1) mód N}** del siguiente. El EOF que publica el *gateway* viaja junto con los datos: de esa manera, llega al *worker* que el algoritmo de RabbitMQ haya elegido acompañado del total de filas que el *gateway* envió al respectivo tópico. Esa cifra es la *verdad de referencia* contra la cual el anillo decidirá si todos los datos fueron consumidos.

El protocolo se compone de dos rondas que viajan por el anillo:

- **Ronda de colecta.** El *worker* que recibió el EOF se declara *iniciador*, guarda el total esperado y arranca un mensaje con su propio conteo de filas procesadas para ese cliente. El mensaje recorre el anillo, y cada *worker* intermedio simplemente suma su conteo acumulado al valor que viene y reenvía al siguiente. Los *workers* intermedios no necesitan recordar el mensaje después de reenviarlo: su único aporte es agregar su parte a la suma parcial. Cuando el mensaje vuelve al iniciador, éste compara el total acumulado contra el total esperado. Si los números coinciden, dispara la ronda de envío. Si no coinciden, el iniciador republica el mismo EOF sobre la cola de la cual lo recibió, para que eventualmente algún *worker* (puede ser él mismo u otro) reinicie la colecta. La hipótesis subyacente es que, si la cuenta no dio, algún *worker* todavía tenía filas en *buffer* sin procesar, y darles una vuelta más les concede el tiempo necesario para drenar esos *buffers* y converger a la cuenta correcta.
- **Ronda de envío.** El iniciador *flushes* sus datos acumulados a la cola correspondiente junto con un EOF y envía un segundo mensaje por el anillo. Cada *worker* intermedio, al recibirlo, hace lo mismo: *flushes* sus datos acumulados con su propio EOF, borra el estado local de los datos que *flushes* y reenvía. Cuando el mensaje vuelve al iniciador, éste da por finalizada la ronda. El componente posterior espera N EOFs por cliente, los recibe gracias a la combinación de los *flushes* de cada *worker* y, recién entonces, procesa la consulta para ese cliente.

De esta manera, el sistema garantiza que ningún componente posterior procese datos de un cliente hasta que todos los *workers* hayan *flushado* sus aportes para ese cliente, lo cual asegura la consistencia y completitud del resultado.

5.2. Garantía de entrega confiable

La garantía de entrega confiable descansa sobre el hecho de que el EOF llega con el total de filas que el cliente envió (para ese tópico). Mientras ese total no coincida con la suma de lo que reportan los *workers*, el sistema se niega a hacer un flush de los datos parciales y republica el EOF. Cada reintento le concede a los *workers* “lentos” más tiempo para drenar sus *buffers*, y el sistema converge eventualmente al resultado correcto. Esto es especialmente importante para consultas que requieren procesar grandes volúmenes de datos, ya que permite manejar situaciones donde algunos *workers* resultan más lentos que otros sin comprometer la integridad del procesamiento.

5.3. Trade-offs

- **Mensajes del protocolo.** Siendo N la cantidad de *workers* en el anillo, este utiliza $2N$ mensajes por cliente en el caso optimista. La ronda de colecta consume N saltos para que el mensaje vuelva al iniciador, y la ronda de envío otros N saltos. El costo es lineal en la cantidad de *workers* del grupo.
- **Costo del reintento.** En presencia de transacciones en *buffer* cuando arranca la primera colecta, el conteo no cierra y se requiere una vuelta de N mensajes adicionales por reintento. Si un mensaje en vuelo se atrasa demasiado, pueden producirse varias vueltas de reintento, lo cual agrega latencia. Sin embargo, esa latencia es el precio de garantizar la entrega confiable sin perder datos, y resulta un compromiso razonable dado que el sistema converge eventualmente al resultado correcto.
- **Frente a un esquema centralizado.** Una alternativa considerada y descartada fue centralizar el conteo en un nodo coordinador mediante confirmaciones explícitas de cada *worker*. Esa alternativa multiplica la cantidad de mensajes en función del tamaño del *batch* y convierte al nodo coordinador en cuello de botella, además de introducir un punto de falla adicional. El anillo, en cambio, distribuye la responsabilidad y mantiene el costo acotado.

En las secciones siguientes, cuando expliquemos cada consulta Q1–Q5 y aparezcan los anillos de control en los diagramas de robustez, se asume que este protocolo garantiza la coherencia del procesamiento.

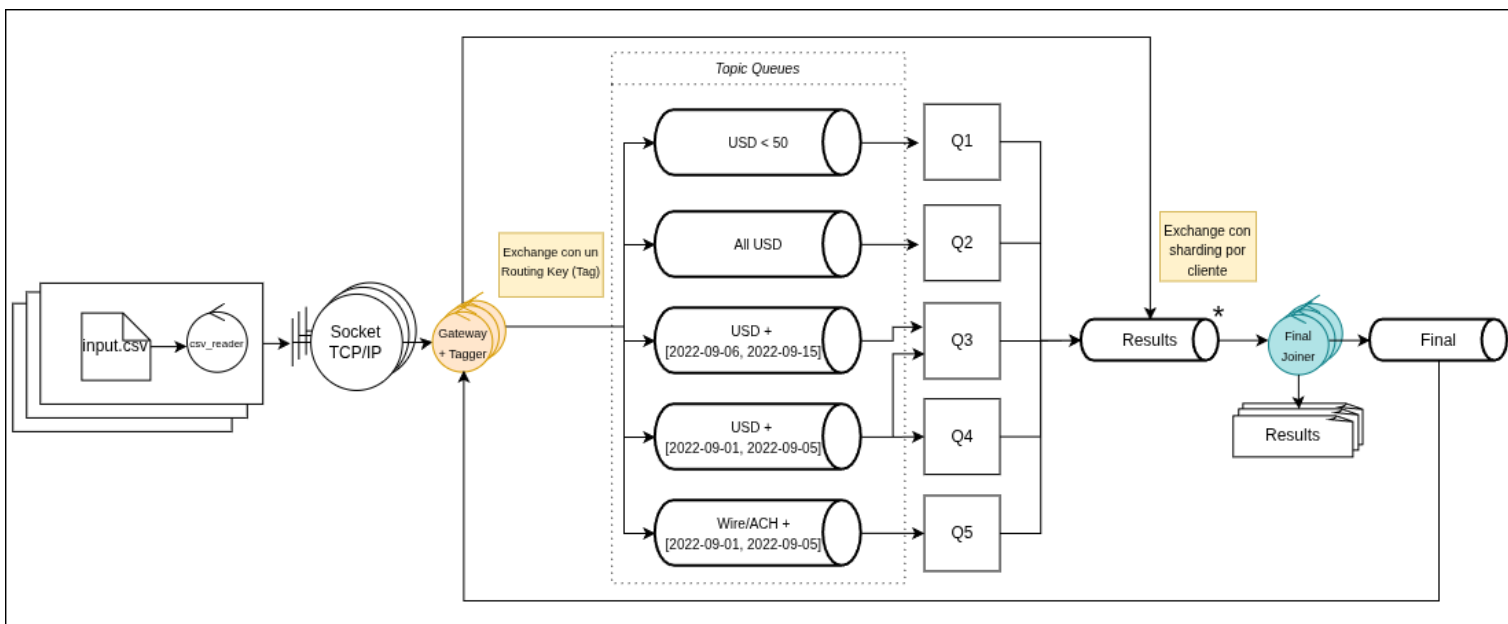
6. Vista Física y de Despliegue

En esta sección describimos cómo se materializan los nodos lógicos definidos en el DAG sobre los recursos físicos del sistema. Se presentan, primero, los diagramas de robustez, que muestran las colas, *exchanges* y *workers* concretos involucrados en cada consulta y, posteriormente, el diagrama de despliegue, que sitúa estos componentes sobre los nodos físicos.

6.1. Diagrama de Robustez

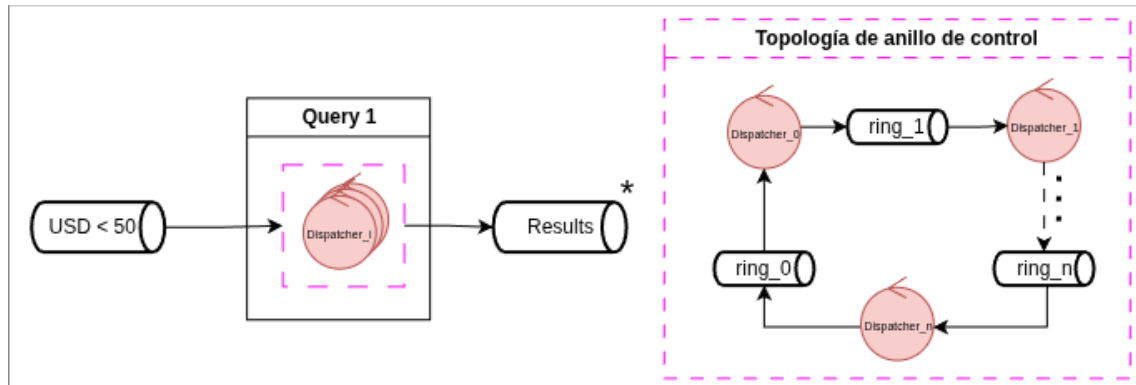
A continuación se presenta la vista general del sistema y, posteriormente, el detalle de cada consulta Q1–Q5. La nomenclatura es uniforme en todos los diagramas: los cilindros representan colas o *exchanges* de RabbitMQ, los controladores encerrados en un cuadrado punteado en fucsia representan grupos de *workers* idénticos coordinados mediante topología de anillo (cuyo protocolo se describió en la sección anterior). El asterisco (*) sobre una cola indica que, en realidad, representa un exchange con múltiples colas, una por cada instancia del worker siguiente. Estas colas son alimentadas por un worker previo al Exchange que se encarga de aplicar una función de hash (sharding) sobre una clave específica (por ejemplo, el ID del cliente, el formato de pago o la cuenta bancaria). Esto garantiza que todos los eventos asociados a una misma entidad sean enrutados y encolados siempre en la misma partición, permitiendo mantener un estado consistente en memoria sin necesidad de locks distribuidos.

- **Vista general.** El cliente se conecta al *Gateway + Tagger* mediante un socket TCP/IP. El *gateway* actúa como *router* hacia un *Topic Exchange* de RabbitMQ, en el cual cada cola está suscrita a un *routing key* específico que captura las transacciones relevantes para esa consulta. Las cinco *Topic Queues* alimentan, respectivamente, las consultas Q1 a Q5. Cada consulta emite sus respuestas hacia un *exchange Results* con *sharding* por cliente, del cual el *Final Joiner* consume hasta haber recibido las cinco respuestas de cada cliente. Una vez consolidada la respuesta, la encola en *Final*, desde donde el *gateway* la recoge para enviarla por el mismo socket TCP del cliente.

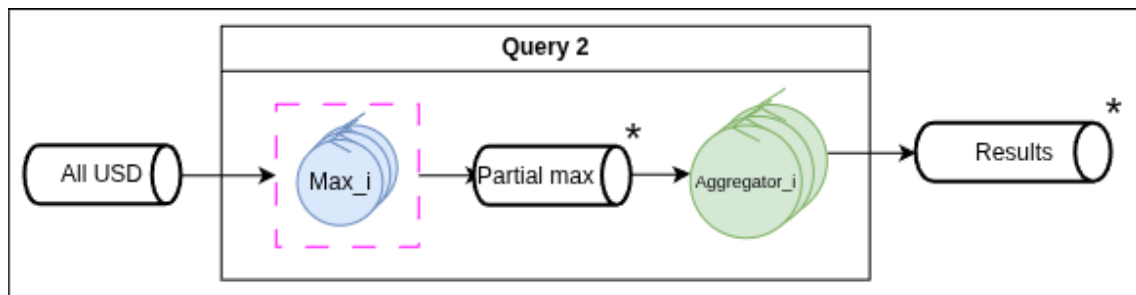


- **Query 1 (Micropagos).** Por su simplicidad, esta consulta se resuelve con un único grupo de *workers* idénticos (*Dispatchers*) en topología de anillo. Cada *Dispatcher* consume directamente del *Topic Queue* USD <50, proyecta la tupla y la encola en *Results*. El recuadro a la derecha del diagrama explicita la topología del anillo de control: cada *Dispatcher* *i* consume

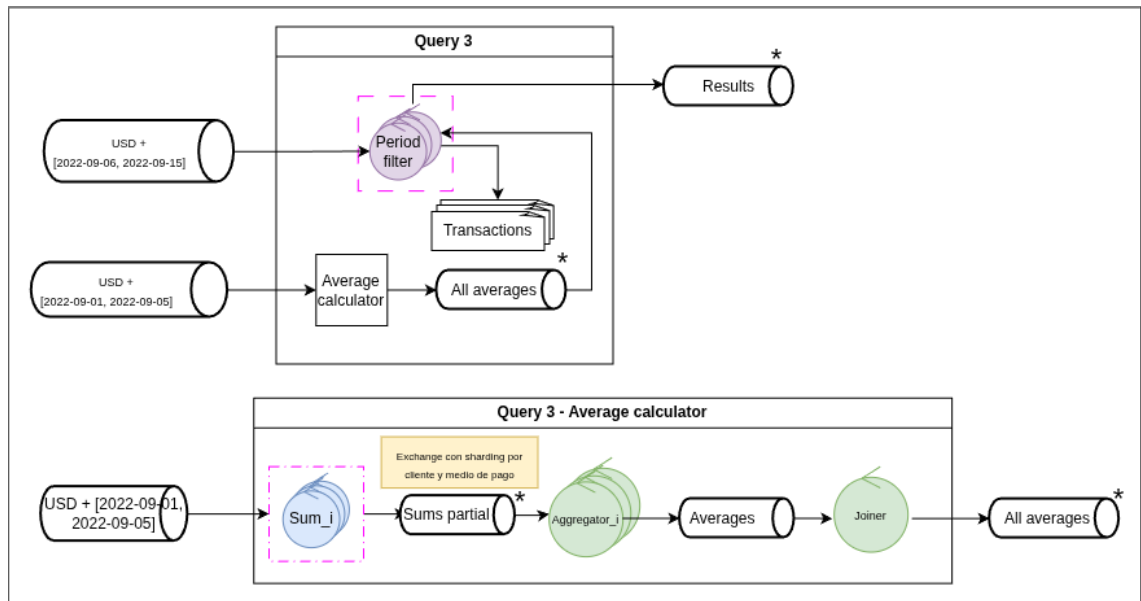
de su cola privada $ring_i$ y publica a $ring_i\{(i+1) \bmod n\}$ para coordinar el cierre por EOF descrito previamente.



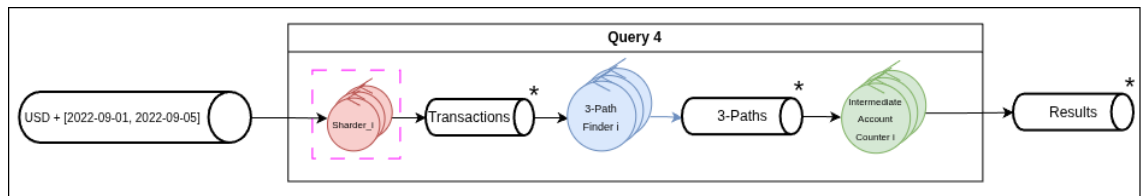
- **Query 2 (Máximos por Entidad).** La consulta Q2 introduce un patrón que se repetirá en otras consultas: una primera etapa de *reducer* local con anillo de EOF (*Max_i*) seguida de una etapa de *aggregator* con *sharding* por cliente (*Aggregator_i*). Los *Max_i* consumen del *Topic Queue* All USD, mantienen el máximo local por banco y, al cierre del cliente, emiten sus parciales al *exchange Partial max*. Los *Aggregator_i* consumen los parciales del cliente que les fue asignado por *sharding*, computan el máximo global por banco y publican el resultado en *Results*.



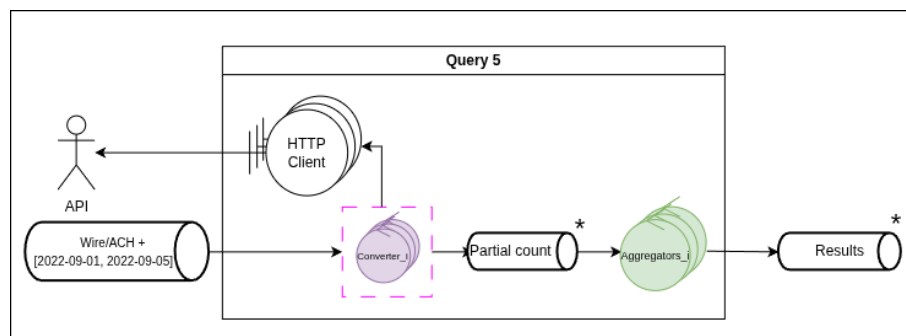
- **Query 3 (Anomalías Históricas).** La consulta está compuesta por dos sub-ramas que se sincronizan internamente. La sub-rama del *Average Calculator* consume del *Topic Queue* del período de referencia, y a través de un grupo *Sum_i* en anillo más un *Aggregator_i* con *sharding* por cliente y formato de pago, calcula el promedio. Un *Joiner* interno consolida estos promedios parciales y los publica en *All averages*, garantizando que el *Period filter* disponga del promedio completo por formato de pago antes de aplicar el filtro. La sub-rama del *Period filter* consume del *Topic Queue* del período analizado y persiste las transacciones en disco; en cuanto recibe el promedio, recorre las transacciones almacenadas y emite a *Results* aquellas cuyo monto sea inferior a un centésimo del promedio del formato de pago correspondiente.



- **Query 4 (Scatter-Gather).** La consulta Q4 está compuesta por tres etapas en *pipeline* comunicadas mediante *exchanges* dedicados. El *Sharder_i* consume del *Topic Queue* y emite cada transacción duplicada al *exchange Transactions* (una copia *shardeada* por cuenta de origen y otra por cuenta de destino). El *3-Path Finder_i* consume de *Transactions* y emite caminos sospechosos al *exchange 3-Paths*, *shardeados* por el par (**origen, destino**). Finalmente, el *Intermediate Account Counter_i* consume de *3-Paths* y emite a *Results* aquellos pares con exactamente cinco cuentas intermediarias distintas. El detalle del algoritmo del *3-Path Finder* y un ejemplo de ejecución se presentan en la *Vista de Procesos*.



- **Query 5 (Micropagos Internacionales).** La consulta Q5 incorpora una particularidad respecto a las anteriores: depende de un servicio externo. Los *Converter_i* consultan, vía cliente HTTP, la API de Frankfurter para obtener (y guardar en memoria) las cotizaciones diarias necesarias para la conversión a USD. Una vez convertido el monto, se incrementa un contador parcial sólo si el monto resulta menor a 1 USD. Al cierre del cliente, los parciales se emiten al *exchange Partial count*, y el *Aggregator_i* consolida la suma final por cliente y la publica en **Results**.



Decidimos realizar una única consulta por nodo *Converter* a la API al iniciar los nodos y almacenar los resultados en memoria. Esta decisión se basa en que verificamos que la API admite múltiples consultas simultáneas, lo cual nos evita implementar la lógica de envío de respuestas a través del anillo.

Asimismo, observamos que es factible almacenar todos los resultados en memoria, dado que los volúmenes de datos son razonables. Por ejemplo, una consulta para la conversión de todas las monedas a USD durante un período de un año ocupa aproximadamente 3.5 MB, mientras que para el período solicitado en el trabajo práctico el tamaño es de alrededor de 40 kB.

Como se observa:

```
1 $ curl -H "Accept: application/x-ndjson" "https://api.frankfurter.dev/v2/rates?from=2022-01-01&to=2022-12-31&base=USD" -o
  rates_2022.ndjson
2 % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
3           %             %       Dload  Upload   Total   Spent    Left   Speed
4 100 3067k    0 3067k    0     0   275k      0  --:--:--  0:00:11 --:--:-- 359k
```

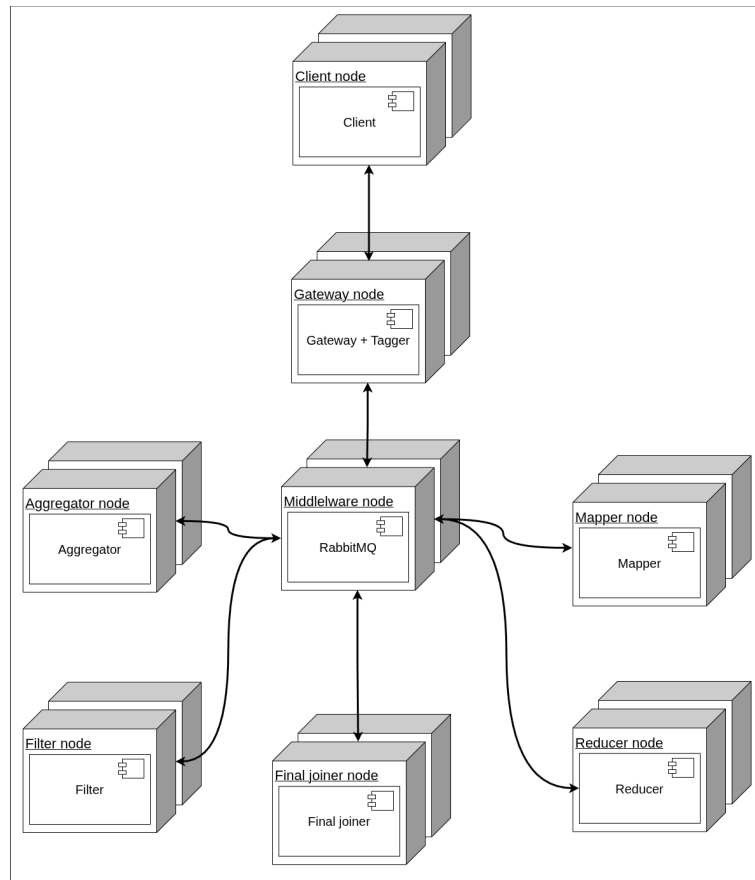
Listing 1: Estadísticas de la solicitud a la API para un período de un año entero

```
1 $ curl -H "Accept: application/x-ndjson" "https://api.frankfurter.dev/v2/rates?from=2022-09-01&to=2022-09-05&base=USD" -o
  rates_period.ndjson
2 % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
3           %             %       Dload  Upload   Total   Spent    Left   Speed
4 100 39708    0 39708    0     0  32261      0  --:--:--  0:00:01 --:--:-- 32256
```

Listing 2: Estadísticas de la solicitud a la API para el período solicitado en el trabajo práctico

6.2. Diagrama de Despliegue

El siguiente diagrama presenta la organización física de los componentes sobre nodos de cómputo. Cada nodo se aloja en un contenedor Docker independiente, y la comunicación entre ellos atraviesa el nodo de *middleware*, que ejecuta el *broker* RabbitMQ. Esta indirección genera la transparencia que requiere un sistema basado en grupos: los *workers* no se conocen entre sí, sino que se acoplan únicamente a las colas y *exchanges* sobre los cuales producen o consumen.

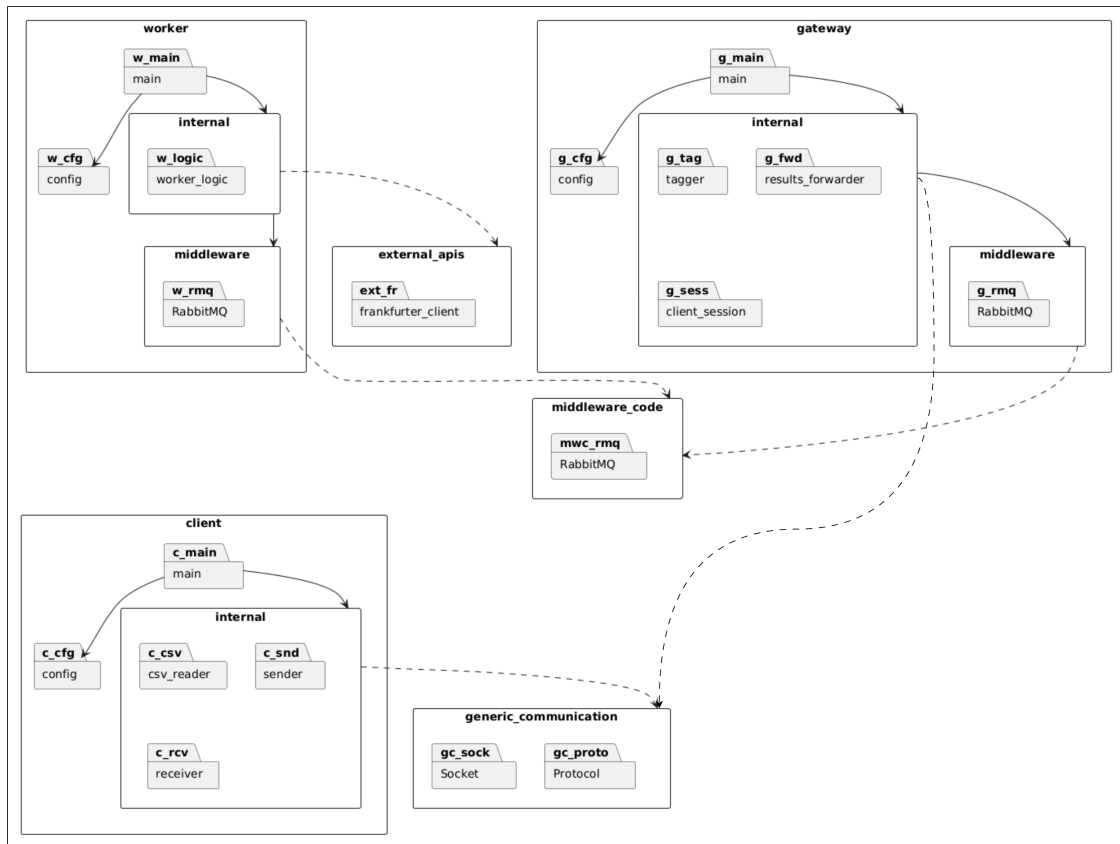


Los nodos del sistema se agruparon, a fines del diagrama, según el rol arquitectónico que desempeñan: *Filter*, *Mapper*, *Reducer*, *Aggregator* y *Final Joiner*. Cada uno de estos grupos puede escalarse horizontalmente de manera independiente, replicando el contenedor del *worker* correspondiente y sumándolo al anillo de control de su grupo (en los casos en que aplica). El nodo del *gateway* es el único punto de contacto con los clientes y, por la simplicidad de su lógica de *tagging*, no constituye en general un cuello de botella. Por simplicidad gráfica los nodos del mismo tipo se agrupan, aún cuando no realicen exactamente la misma tarea en cada consulta (por ejemplo, los *aggregators* de Q2, Q3, Q4 y Q5 ejecutan agregaciones distintas).

7. Vista de Desarrollo

En esta sección presentamos la organización del sistema, agrupado en paquetes que se corresponden con las unidades de despliegue y con los componentes transversales reutilizados por más de una unidad.

7.1. Diagrama de Paquetes



El diagrama anterior se estructura en torno a tres unidades principales — **client**, **gateway** y **worker** — y tres paquetes transversales que son reutilizados por las anteriores: **generic.communication**, **middleware.code** y **external.apis**.

- **client**. Contiene el ejecutable que ejecuta el cliente del sistema. Su **main** delega en un módulo **internal** que agrupa la lectura del CSV de entrada (**csv_reader**), la emisión de *batches* hacia el *gateway* (**sender**) y la recepción de los resultados consolidados (**receiver**). El paquete **config** concentra los parámetros configurables (tamaño de *batch*, dirección del *gateway*, ruta del archivo de entrada, etc.). El cliente depende del paquete transversal **generic.communication** para todo lo relativo al socket TCP y al protocolo aplicativo.
- **gateway**. Aloja el *gateway* y el *tagger*. Su **main** delega en un módulo **internal** con tres responsabilidades bien diferenciadas: el *tagger* se ocupa de evaluar el contenido de cada transacción y asignarle el *routing key* apropiado para el *Topic Exchange*; el *results_forwarder* consume de la cola **Final** y envía los resultados al cliente correspondiente; y **client_session** mantiene la asociación entre identificador de cliente y socket TCP activo. La comunicación con RabbitMQ pasa por el módulo **middleware** interno, que a su vez se apoya en el paquete transversal **middleware.code**. La capa de socket y protocolo, al igual que en el cliente, depende de **generic.communication**.
- **worker**. Es la unidad genérica que aloja la lógica específica de cada consulta Q1–Q5 y de los componentes auxiliares (*Sharder*, *3-Path Finder*, *Aggregators*, *Final Joiner*). El módulo **worker.logic** encapsula el comportamiento particular del *worker*, mientras que la comunicación con RabbitMQ se canaliza por el módulo **middleware** interno. Cuando un *worker* requiere acceder a servicios externos (caso del *Converter* de Q5), el módulo **worker.logic** se apoya en el paquete transversal **external.apis**.

- **generic_communication.** Paquete transversal que provee dos abstracciones reutilizadas por `client` y `gateway`: `Socket`, que encapsula la operatoria de bajo nivel del socket TCP, y `Protocol`, que define el formato de los mensajes intercambiados entre cliente y `gateway` (*batches* de transacciones, señal de EOF y respuestas consolidadas).
- **middleware_code.** Paquete transversal que abstrae el cliente AMQP de RabbitMQ utilizado por todo lo que publica o consume de colas. Concentrar esta dependencia en un único paquete permite que el resto del código no dependa directamente de la librería elegida y, eventualmente, sustituirla por otra implementación sin afectar la lógica de negocio.
- **external_apis.** Paquete transversal que aloja clientes HTTP hacia servicios externos. En el alcance actual, contiene únicamente el `frankfurter_client`, utilizado por el `Converter` de Q5 para obtener las cotizaciones diarias necesarias para la conversión a USD.

La separación entre paquetes internos a cada unidad y paquetes transversales se diseñó con dos objetivos: por un lado, evitar la duplicación de código en componentes que comparten necesidades (cliente y `gateway` comparten el protocolo TCP, `gateway` y `worker` comparten el cliente AMQP); por el otro, garantizar que las decisiones tecnológicas concretas (la librería de RabbitMQ, la implementación HTTP) queden contenidas en un único paquete y no se filtren al resto del sistema.

8. Planificación y Organización

En la presente sección consignamos el listado de tareas que componen el desarrollo del sistema, junto con la división tentativa entre los integrantes del equipo. La granularidad de las tareas se eligió de modo de coincidir, en líneas generales, con los componentes identificados en las vistas anteriores, lo que facilitará la integración y reducirá las dependencias entre integrantes durante la implementación.

8.1. Listado de Tareas a Ejecutar

- **Cliente.** Implementación del lector de CSV, armado y envío de *batches* por TCP, manejo de la señal de EOF hacia el `gateway` y recepción de los resultados consolidados.
- **Gateway + Tagger.** Implementación del servidor TCP, asociación de `ClientID` a sockets, lógica de *tagging* por contenido de transacción, contadores por *topic* para inyectar el total esperado en los EOFs y reenvío de los resultados consolidados al cliente correspondiente.
- **Protocolo cliente ↔ gateway.** Definición del formato de *batches*, señal de EOF y mensajes de respuesta. Implementación dentro del paquete transversal `generic_communication`.
- **Protocolo de anillo de EOF.** Implementación de las dos rondas (colecta y envío) descritas en la sección *Manejo de EOF*, así como el mecanismo de reintento ante discrepancia con el total esperado.
- **Workers de Q1.** Implementación del *Dispatcher* con anillo de control y emisión a **Results**.
- **Workers de Q2.** Implementación del *Max* local con anillo de control y del *Aggregator* con *sharding* por cliente.
- **Workers de Q3.** Implementación de las dos sub-ramas: *Sum* con anillo de control, *Aggregator* y *Joiner* para el *Average Calculator*; y *Period filter* con persistencia local de transacciones a la espera del promedio.
- **Workers de Q4.** Implementación del *Sharder*, del *3-Path Finder* (con su estructura interna de cuentas *Entrada/Salida*) y del *Intermediate Account Counter* con *sharding* por par (*origen*, *destino*).

- **Workers de Q5.** Implementación del cliente HTTP hacia la API de Frankfurter (paquete `external_apis`), del *Converter* con anillo de control y del *Aggregator* con *sharding* por cliente.
- **Final Joiner.** Implementación del consumidor del *exchange Results* con *sharding* por cliente, gestión del estado parcial por cliente y emisión de la respuesta consolidada a **Final** cuando se reciben los EOFs de las cinco ramas.
- **Despliegue.** Construcción de las imágenes Docker de cada unidad (**client**, **gateway**, **worker**), *compose* de la topología completa con RabbitMQ y configuración de las réplicas iniciales por grupo.
- **Pruebas funcionales.** Verificación de los resultados de cada consulta contra el *notebook* patrón provisto por la cátedra.

8.2. División del Trabajo entre Integrantes

Tarea	Encargado
Cliente	Juan Martín de la Cruz
Gateway + Tagger	Ian Klaus von der Heyde
Protocolo cliente ↔ gateway	Ian Klaus von der Heyde
Protocolo de anillo de EOF	Ian Klaus von der Heyde
Workers de Q1 (Micropagos)	Juan Martín de la Cruz
Workers de Q2 (Máximos)	Agustín Altamirano
Workers de Q3 (Promedios)	Juan Martín de la Cruz
Workers de Q4 (Scatter-Gather)	Agustín Altamirano
Workers de Q5 (Cotizaciones API)	Ian Klaus von der Heyde
Final Joiner	Agustín Altamirano
Despliegue (Docker/Compose)	Agustín Altamirano, Ian Klaus von der Heyde, Juan Martín de la Cruz
Pruebas funcionales	Juan Martín de la Cruz